

Fundación Fulgor

Curso Protocolos de Comunicaciones de Alta Velocidad 2026

Informe Trabajo práctico 1 y Trabajo práctico 2

Diseño y Testeo de sistema de desplazamiento de leds y de generador PRBS para detección de errores en comunicación

Guerrero Pozzi, Facundo Esteban

Profesores

Facundo Sposetti

Jorge Manuel Finochietto

Índice

Introducción **3**

TRABAJO PRÁCTICO 1 **3**
- Descripción 3
- Restricciones y decisiones de diseño 3
- Arquitectura 4
- Diseño de los módulos 4
  - counter.v 4
  - shift_reg.v 6
  - top_leds.v 8
- Testing 9
- Implementación y Síntesis 10

TRABAJO PRÁCTICO 2 **14**
- Descripción 14
- Restricciones y decisiones de diseño 14
- Arquitectura 15
- Diseño de los módulos 16
  - lfsr.v (generador) 16
  - checker.v 18
  - top_lfsr.v 21
- Testing 22
  - Resumen de tests 25
  - Test 4 Test de Fronteras (Thresholds) 26
  - Test 6 Resiliencia ante Resets randoms 28
  - Test 7 Cambio Dinámico de Semilla 31
- Implementación y Síntesis 33
- Observaciones y aspectos a mejorar 34

Conclusiones **34**
ANEXO **35**

Introducción

En este informe se presentará lo realizado en los trabajos prácticos 1 y 2 del curso de Protocolos de Comunicaciones de Alta Velocidad. En el trabajo práctico 1 se desarrolla un sistema de desplazamiento de LEDs, mientras que en el trabajo práctico 2 se implementa un sistema de generación y verificación de números pseudoaleatorios para la detección de errores.

Se desarrolla el contexto, las decisiones de diseño y las pruebas realizadas para cada trabajo. Finalmente, se presentan las conclusiones obtenidas a partir de los resultados y del proceso de diseño.

TRABAJO PRÁCTICO 1

Descripción

En este trabajo práctico una vez estudiada la parte teórica introductoria de diseño RTL en verilog, se paso al desarrollo de código necesario para la implementación un sistema de shifteo de leds, para poder ser cargado en una FPGA Artix-7 XC7A100T tanto del diseño sintetizable como los tests correspondientes.
Consiste en un módulo contador de 32 bits, con cuatro límites superiores de cuenta seleccionables mediante los switches de la placa, el cual al contar hasta dicho límite seleccionado debe enviar una señal a otro módulo llamado Shift Register, que se encargará de habilitar el led correspondiente realizando un shifteo circular hacia la derecha.

Restricciones y decisiones de diseño

Se partió de ciertas restricciones a respetar en el diseño:

- Se utilizan los cuatro switches incluidos en la placa de donde:
    - El sw[0] se utilizará para la habilitación del contador
    - Los sw[2:1] se emplearán para seleccionar alguno de los cuatro posibles límites de cuenta.
    - el sw[3] se utilizará para elegir el color de los leds RGB.

- Se debe poder cambiar el límite del contador en cualquier momento.
- ***i_reset*** es el reset del sistema, el cual pone a cero el contador e inicializa el shift register a la izquierda.

Arquitectura

El módulo top_leds instancia dos bloques internos , el counter (que contiene la descripción del contador) y shift_reg (módulo encargado de realizar la habilitación del led correspondiente para el shifteo). La parte del VIO no se utilizará por poder contar con la placa física y se utilizará el ILA para ver el valor de led activo en vivado.

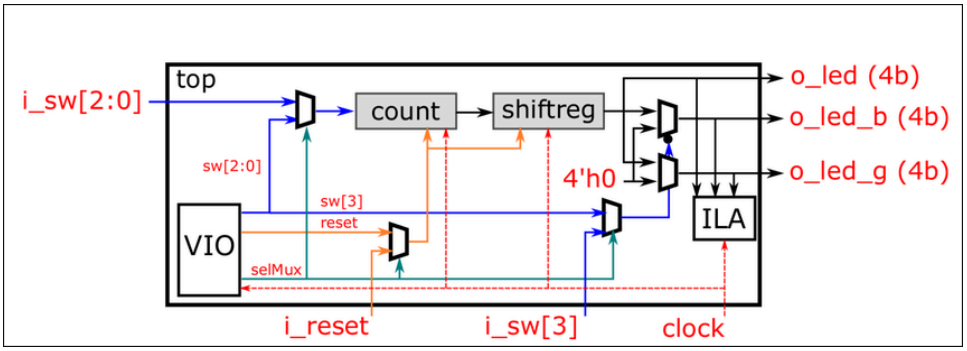


Imagen 1.1 Diagrama en bloques

Diseño de los módulos

counter.v

Implementa el contador de 32 bits con cuatro límites de cuenta seleccionables mediante i_sel_count_limit. Los límites son potencias de 2 consecutivas, de forma que cada uno duplica exactamente al anterior, generando una progresión geométrica de velocidades:

```
module counter
    #(
        parameter NB_COUNT_LIMIT = 2
    )
    (
        // --> outputs
        output wire     o_shift,  //señal para que shiftee el shifter

        // --> inputs
        input wire      i_enable,
        input wire [NB_COUNT_LIMIT - 1 : 0] i_sel_count_limit, //input con el limite de la cuenta
        input wire i_reset ,
        input wire clock
    );
```

Imagen 1.2 Entradas y salidas del contador

Parámetros

Parámetro	Comportamiento
NB_COUNT_LIMIT	Cantidad de bits del seleccionador de límite de cuenta.

Entradas

Entrada	Comportamiento
i_enable	Señal que habilita el contador (se controlará con *i_sw[0]*
i_sel_count_limit	Input con el límite de la cuenta a seleccionar
i_reset	Señal de reset asíncrono
clock	Señal de clock

Salidas

Salida	Comportamiento
o_shift	Señal hacia shift_reg para que shiftee el led

counter es un registro que almacena el valor del contador.
counter_next es el valor siguiente del contador.

La asignación se realiza a **counter_next** de manera que al ser mayor o igual al límite el contador se reinicia a 0. De esta forma

```
reg  [NB_COUNTER - 1 : 0] counter                    ; //variable del contador
wire [NB_COUNTER - 1 : 0] counter_next               ; //variable del contador siguiente
//asignacion, asigna el siguiente valor del contador
// a 0 si se supera el limite seleccionado
// o a contador + 1 si no se supera el limite seleccionado
assign counter_next = ((i_sel_count_limit == 2'b00) && (counter >= limit_0)) ? 'd0 :
                      ((i_sel_count_limit == 2'b01) && (counter >= limit_1)) ? 'd0 :
                      ((i_sel_count_limit == 2'b10) && (counter >= limit_2)) ? 'd0 :
                      ((i_sel_count_limit == 2'b11) && (counter >= limit_3)) ? 'd0 :
                                                                               counter + 1'b1;
```

Imagen 1.3 Asignacion del valor siguiente del contador

Esto lo que permite es poder hacer el cambio de límite en cualquier momento y tener un comportamiento adecuado. Si la condición de asignación fuera *counter == límite* por ejemplo, podría darse el caso de asignarle un límite de cuenta mas chico que su valor actual, por ejemplo donde el contador esté entre el límite 3 y el 4 y cambiarlo al límite 2

(más chico), esto provocaría que el contador no llegue al límite, teniendo que esperar a su desbordamiento para que cambie el led (lo que no es un comportamiento deseado).

A la variable del contador se le asigna su valor siguiente por cada flanco positivo de clock. Finalmente se pone en alto **o_shift** cada vez que el contador llega a 0.

```
    always@(posedge clock or negedge i_reset)
        begin
            if(!i_reset) //si se presiona el reset (activo por bajo)
            begin
                counter <= 'd0;
            end
            else if (!i_enable) //si no esta habilitado el contador, se mantiene el valor actual (enable activo por alto)
            begin
                counter <= counter;
            end
            else
            begin
                counter <= counter_next; //si esta habilitado el contador, se actualiza al siguiente valor
            end
        end

    assign o_shift = (counter == 'd0);

endmodule
```

Imagen 1.4 Asignación de contador y o_shift

shift_reg.v

Este módulo es el encargado de realizar la activación del led correspondiente para el shifteo circular derecho.

```
module shift_reg
  #(
    parameter   NB_SHIFT_REG = 4
  )
  (
    //----> Outputs
     output wire [NB_SHIFT_REG - 1 : 0]  o_led_enable , //es el led que se ira shifteando
    //----> Inputs
     input  wire                         i_shift      , //señal que viene desde el contador para hacer el shift
     input  wire                         i_enable     ,
     input  wire                         i_reset      ,
     input  wire                         clock
  );
```

Imagen 1.5 Entradas y salidas de shift_reg

Parámetros

Parámetro	Comportamiento
NB_SHIFT_REG	Cantidad de leds

Entradas

Entrada	Comportamiento
i_shift	Señal que viene desde el contador para hacer el shift
i_enable	Habilita el shift_register
i_reset	Reset asíncrono
clock	Señal de clock

Salidas

Salida	Comportamiento
o_led_enable	Es el led que se ira shifteando

En el registro ***shift_register*** se guarda el shifteo circular derecho que se asigna en la linea 27. Finalmente se asigna ese valor a la salida ***o_led_enable***

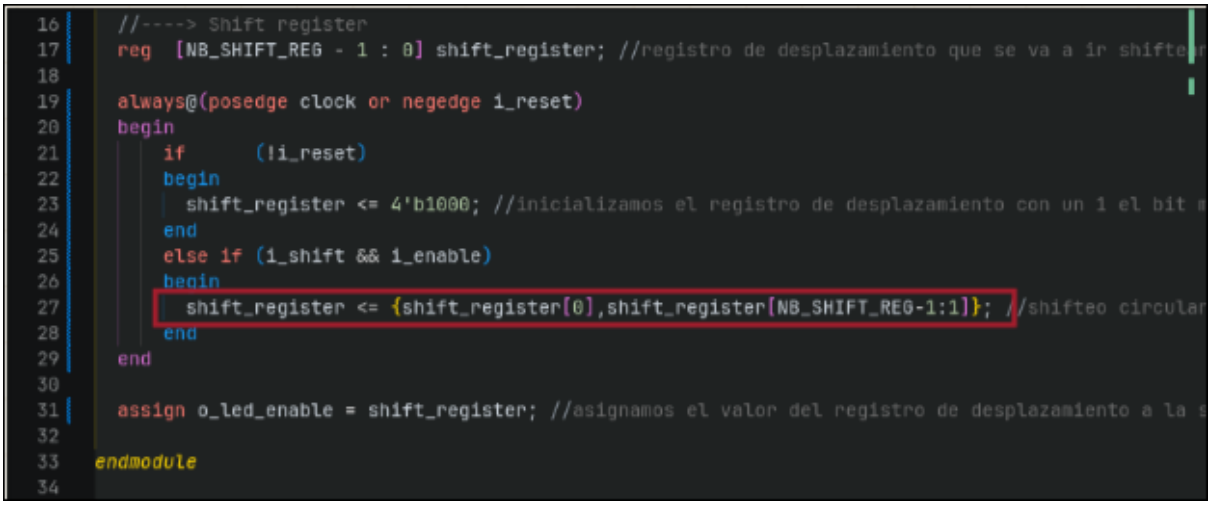


Imagen 1.6 Asignación del shifteo circular derecho

top_leds.v

Integra ambos bloques, conecta ***o_shift*** del contador con ***i_shift*** del shift register, comparte ***i_sw[0]*** como enable de ambos, y agrega la selección de color.



```
counter
u_counter
(  //----> Output
  .o_shift                      (shift    ),
  //----> Inputs
  .i_enable                     (i_sw[0]  ),
  .i_sel_count_limit            (i_sw[2:1]),
  .i_reset                      (i_reset  ),
  .clock                        (clock    )
)                                         ;

shift_reg
u_shift_reg
(
  //----> Outputs
  .o_led_enable                 (led_enable),
  //----> Inputs
  .i_shift                      (shift     ),
  .i_enable                     (i_sw[0]   ),
  .i_reset                      (i_reset   ),
  .clock                        (clock     )
)                                           ;


//----> Outputs
assign o_led    = led_enable;
//elijo entre led blue o green con i_sw[3], y le cargo led_enable o 0
assign o_led_b  = ( i_sw[3]) ? led_enable : 4'b0000   ;
assign o_led_g  = (!i_sw[3]) ? led_enable : 4'b0000   ;
```

Imagen 1.7 Instancia de módulos en top.

Testing

Para realizar el testing se definio un testbench (***top_leds_tb.sv***) en el cual instancia el top, se define una tarea de reset asincrónico que simula el botón presionado por un tiempo random, y se importa los archivos de forma modular de los TESTS (1-6) con *ifdef* según se desee en un *define*.

También para los tests 3 y 4 se instancia un modelo de referencia de comportamiento independiente (***modelo_ideal.sv***) que reproduce el comportamiento esperado del sistema para poder comparar con él de forma continua la salida real del DUT.

Todos los tests se realizaron en systemverilog por mayor comodidad en herramientas de testeo, se priorizo utilizar tiempos o valores random cada vez que fue posible y alrededor de 100 iteraciones por test para probar una gran cantidad de casos con variabilidad.

Test	Qué verifica	Técnica utilizada
TEST1	Estabilidad de ***o_led*** cuando el sistema está deshabilitado (*i_sw[0] = 0*).	Se pone en 0 el enable, y se espera un número aleatorio de ciclos, se chequea que la salida *o_led_enable* no cambie.
TEST2	Gating de clock: el diseño queda estático sin clock y retoma correctamente al reactivarlo.	Se deshabilita el clock, y se compara el estado actual con el previo 50 veces (esperando un momento random por cada comparación), luego se reactiva el clock y se compara el estado actual con el previo (debe cambiar) si son iguales se resta un contador de 100, ya que puede darse la casualidad que justo sea un ciclo completo, pero no 100 veces.
TEST3	Verifica que al cambiar el limite de cuenta, cambie el tiempo de shifteo.	Se parte del conocimiento de cantidad de ciclos de clock para alcanzar cada límite, se cambia el límite y se cuenta la cantidad de ciclos que le toma shiftear desde el contador en 0,, este valor y el esperado deben ser exactamente iguales
TEST4	Robustez ante cambio de límite de velocidad en tiempo de ejecución (no se cuelga).	Cambia el límite a mitad de cuenta y verifica que el LED siga shifteando dentro de límite + límite/2 ciclos. Se hace una comparación paralela con modelo_ideal todo el tiempo.
TEST5	Selección de color correcta, incluso al cambiarla en tiempo real.	Verifica que *o_led_g* / *o_led_b* sean mutuamente excluyentes y coincidan con *o_led* según *i_sw[3].*
TEST6	Integración: arranque, reset y reanudación del conteo.	Se resetea en mitad de funcionamiento y se verifica que los valores vayan a valor de reset, y que luego salgan de ahi shifteando como debe.

Implementación y Síntesis

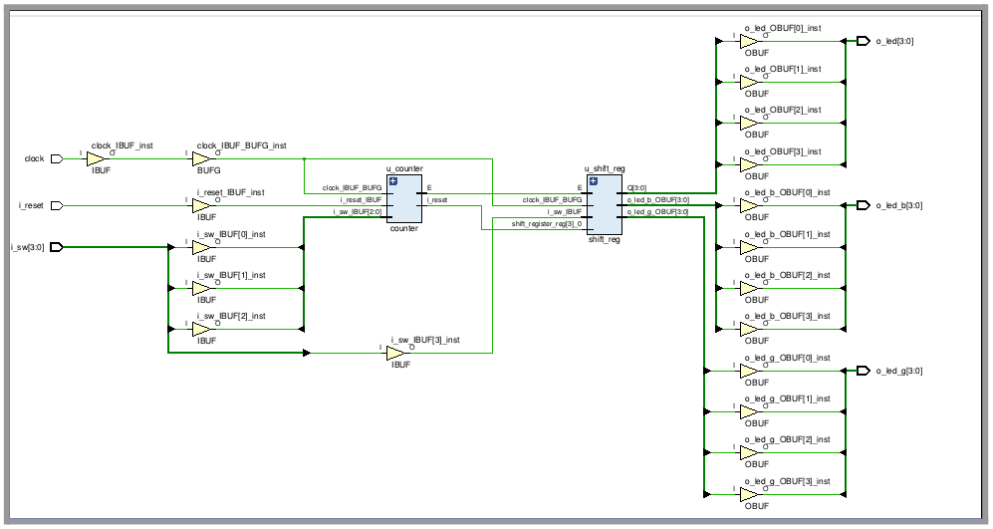


Imagen 1.8 Esquemático resultante.

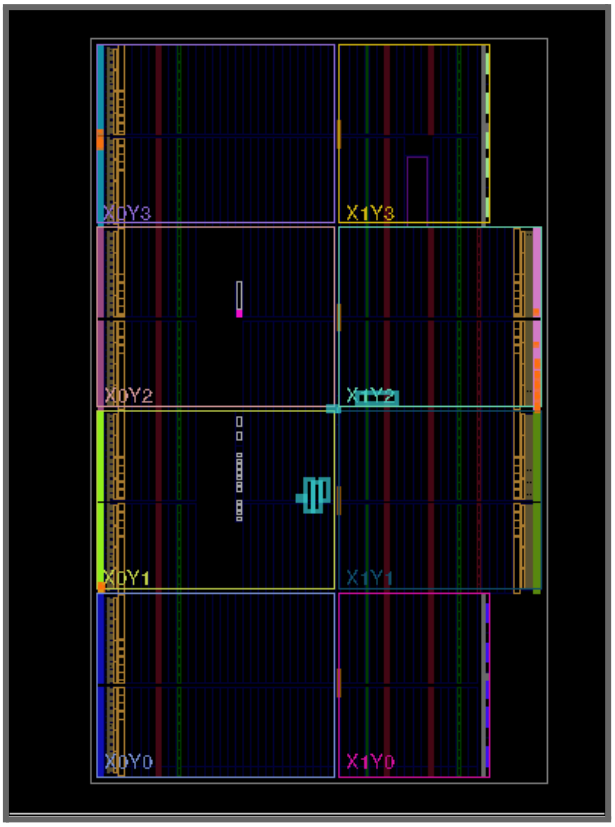


Imagen 1.9 Implementación

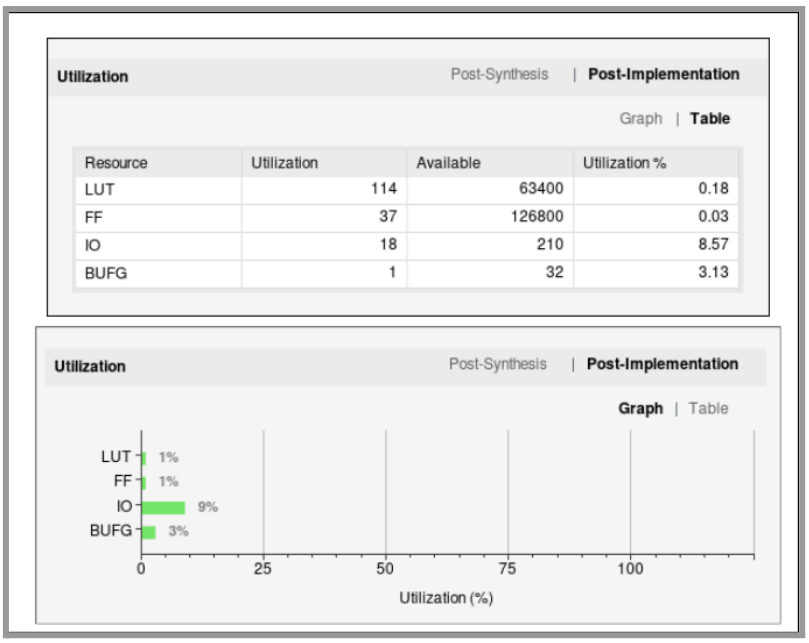


Imagen 1.10 Resumen de recursos utilizados

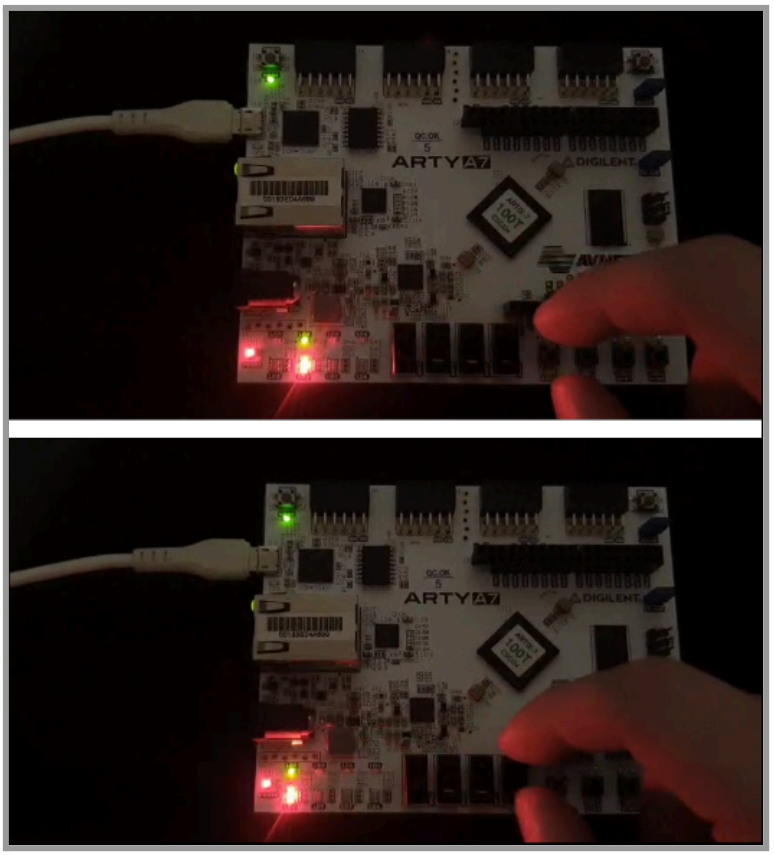


Imagen 1.11 Diseño funcionando en FPGA

Para finalizar se configuró el ILA para ver el comportamiento en vivado. Se puede ver como ahora en total usa mucho más recursos que nuestro diseño de leds.

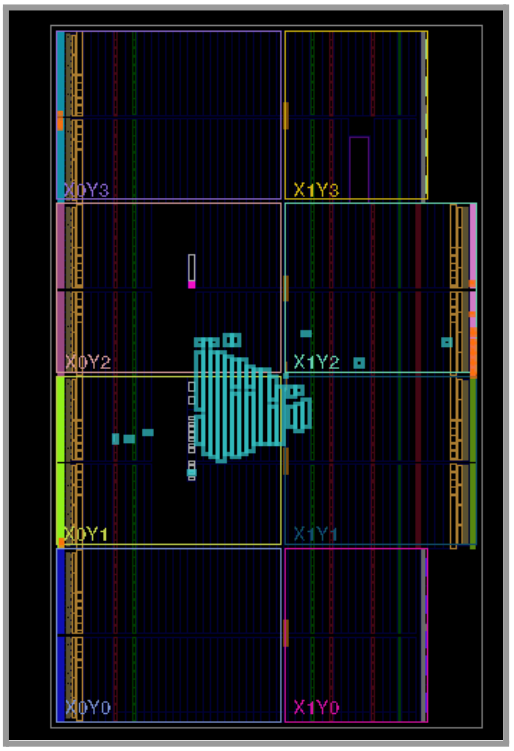


Imagen 1.11 Implementación con ILA.

En un principio se podía ver en pantalla solamente el estado actual del led encendido en ese momento.

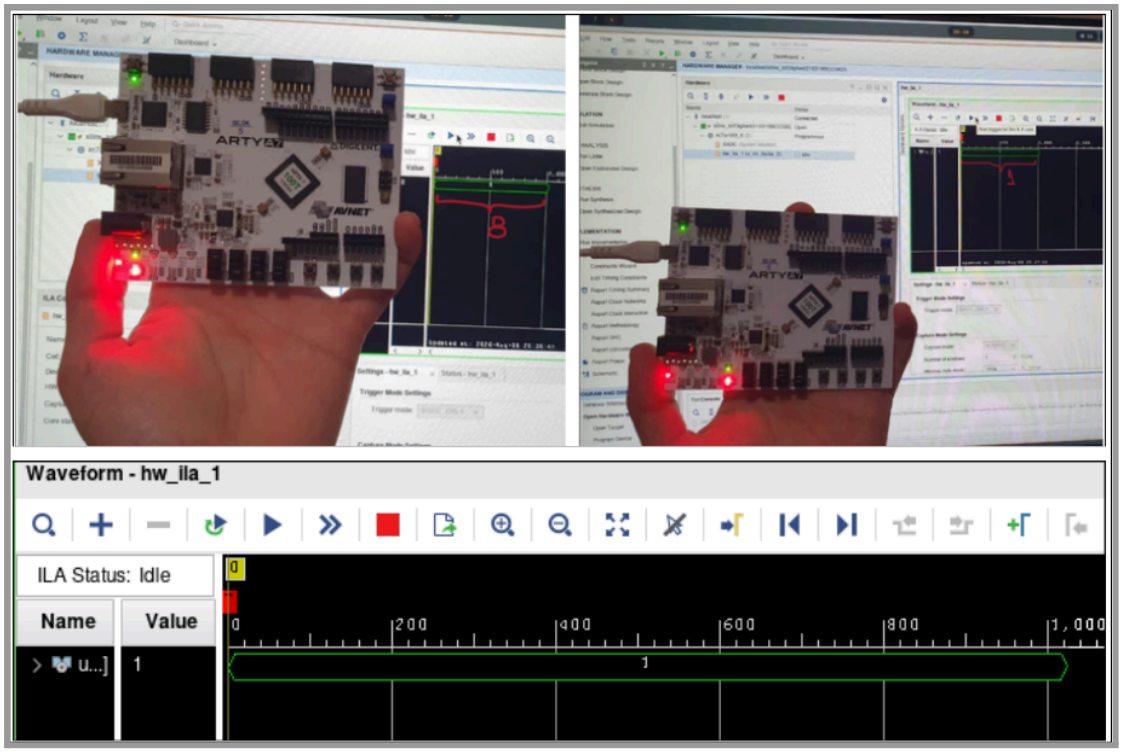


Imagen 1.12 Waveform de ILA

Esto sucede debido a que en el ILA de Vivado uno puede ver hasta 1024 muestras, lo cual para un clock de *100MHz* es un intervalo de tiempo de **0.01024 ms** = ***10240 ns*** *(*Es decir podemos ver 10240 ns de tiempo en la pantalla*)*.
El menor de los límites configurado era de *limit_0 <= 32'h0010_0000* equivalente a **1.048.576** en decimal, el contador partiendo de cero hasta ese límite y un clock de 100MHz le tomaría **≈10,49 ms = 1.049x10^{9} ns** shiftear el led, o sea, podemos ver en pantalla un lapso de 0.01ms pero al led le toma 10.5 ms shiftear.

Para probar se pasó a reducir el valor del límite más chico a *limit_0 <= 32'h0000_0010* equivalente a **16** decimal que le tomaría unos **0.00017 ms** = **170 ns** shiftear el led, como dijimos que podíamos ver **0.01024 ms = *10240 ns*** en pantalla esto nos da lugar a que podamos ver unos ahora unos 60 shifteos siguiendo el patrón 8 4 2 1 que corresponde al shifteo circular derecho de los leds.

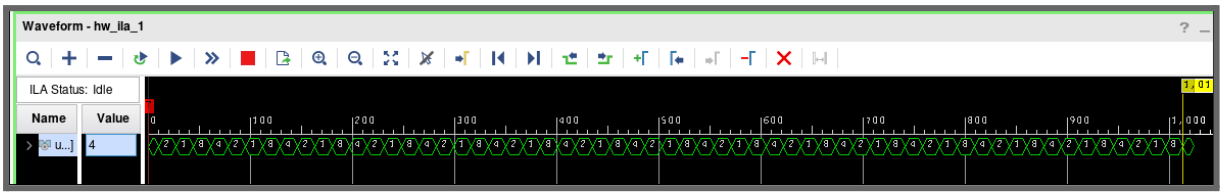


Imagen 1.13 Waveform con 60 shifteos ILA

El código completo del TP1 puede encontrarse en el ANEXO.

TRABAJO PRÁCTICO 2

Descripción

En este trabajo práctico se estudió la utilidad de los generadores y checkers PRBS para detección de errores en la comunicación. Se realizó el diseño y testeo de un generador de LFSR galois junto con el correspondiente checker, definido como una máquina de estados (lock & unlock) con thresholds.

Consiste en un generador de números pseudoaleatorios que sigue una secuencia determinada que depende de su diseño y el valor anterior (el cual se inicializa con una semilla SEED), y un checker el cual es independiente del generador aunque se comunica con él. El generador produce los números de la secuencia y el checker toma el valor recibido para predecir el siguiente (ya que conoce el polinomio generador de antemano), el checker compara constantemente que lo que recibe el generador sea siempre igual a lo que predice (con cierto threshold de tolerancia de fallos). El objetivo es poder determinar que la comunicación sea fiable, y se utilizan estos patrones pseudoaleatorios para acercarse más a lo que puede llegar a ser el payload que interesa enviar por dicho canal.

Restricciones y decisiones de diseño

Se partió de ciertas restricciones a respetar y decisiones a implementar en el diseño:

- El generador es un generador tipo galois de 16 bits.
- El bit de feedback es el bit 16.
- La combinación es aquella que genere la secuencia más larga antes de repetir el valor.
- Hay una señal de valid la cual indica cuando debe generarse el próximo valor de la secuencia.
- Se debe diseñar de forma tal que el checker no dependa del generador, es decir, puede estar a kilómetros de distancia.
- El generador recibe del checker el valor generado y la señal de valid de salida.
- La señal de valid a la salida del generador debe “seguir” al dato en términos de atraso de ciclos.
- El Checker tiene dos estados (lock & unlock).
- Se definen Thresholds para pasar de un estado a otro.
- El checker si está en estado *unlock* utiliza la entrada que le llega del generador para verificar si matchean los datos.
- El checker al pasar a esto de *lock* utiliza su propia lógica interna para calcular el siguiente estado.

Arquitectura

El módulo ***top_lfsr*** instancia dos bloques internos, el generador (***lfsr.v***) (que contiene la descripción del generador) y ***checker.v*** (el cual toma el valor producido por el generador y lo compara con un valor predicho).

El Checker y el generador son independientes aunque están comunicados entre sí. El generador envía el valor generado y la señal de valid que acompaña temporalmente a dicho dato, el checker tiene las salidas correspondientes para informar su estado de lockeo y el valor predicho.

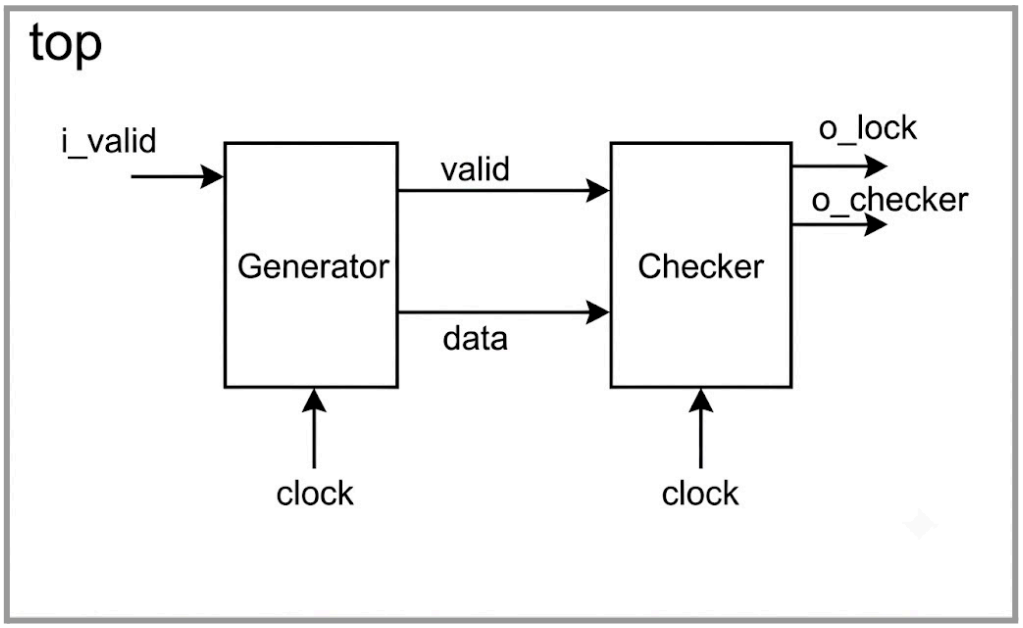


Imagen 2.1 Diagrama en bloques

Diseño de los módulos

lfsr.v (generador)

Implementa el generador LFSR galois de 16 bits, el bit de feedback es el bit más significativo. La combinación es la cual genera la mayor cantidad de datos sin repetir valor, genera una cantidad de 65535 datos antes de que se repita la secuencia.

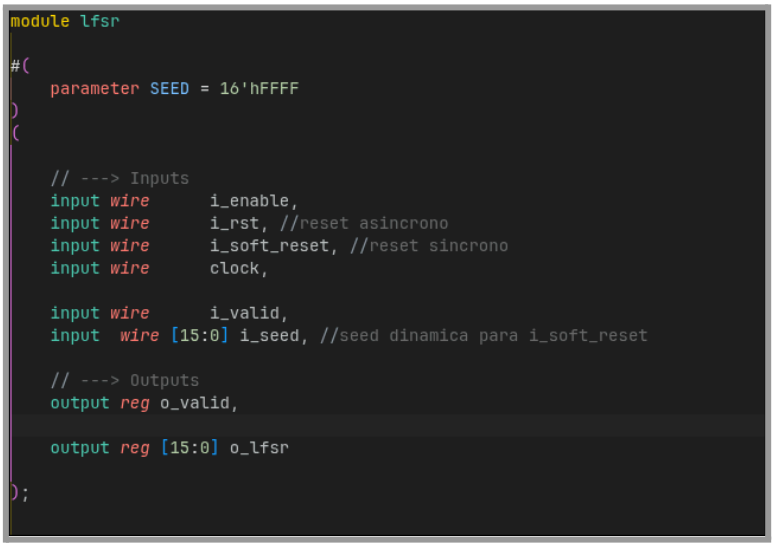


Imagen 2.2 Entradas y salidas del generador

Parámetros

Parámetro	Comportamiento
SEED	Es el valor inicial con el cual comienza la secuencia después del reset.

Entradas

Entrada	Comportamiento
i_enable	Señal que habilita el generador
i_sel_count_limit	Input con el límite de la cuenta a seleccionar
i_rst	Señal de reset asíncrono activo por alto
i_soft_reset	Señal de reset síncrono
clock	Señal de clock
i_valid	Señal de valid que indica la generación del siguiente dato
i_seed	Valor de seed a tomar luego de un soft_reset

Salidas

Salida	Comportamiento
o_valid	Señal de valid alineada temporalmente con el dato de salida.
o_lfsr	Dato de salida del generador

Podemos ver en la siguiente imagen como está formado el diseño del generador, con el bit de feedback asignado al bit más significativo, y utilizando operación XOR para los *taps*.
También es importante señalar las asignaciones hacia *o_valid*, las cuales se realizan para realizar la compensación de latencia con respecto al dato.
Se definen también comportamientos para reset asincrónico *i_reset* donde el primer valor es la semilla SEED definida, y para reset sincrónico donde el valor de la semilla pasa a ser el leído en ese momento en el puerto *i_seed*.

```
    wire feedback;

    always @(posedge clock or posedge i_rst)
    begin
        // ---> Asignamos shifteando, el valor del 1 va al 2 y asi sucesivamente
        // salvo en los que hayamos seleccionado como realimentados
        //donde se le asigna el valor anterior pero XOR bit de feedback
        if(i_rst)
        begin
            o_lfsr <= SEED;
            o_valid <= 1'b0;
        end
        else if (i_soft_reset) begin
            o_lfsr <= i_seed;
            o_valid <= 1'b0;
        end
        else if(i_enable && i_valid)
        begin
            o_lfsr[0] <= feedback;
            o_lfsr[1] <= o_lfsr[0];
            o_lfsr[2] <= o_lfsr[1] ^ feedback;
            o_lfsr[3] <= o_lfsr[2] ^ feedback;
            o_lfsr[4] <= o_lfsr[3];
            o_lfsr[5] <= o_lfsr[4] ^ feedback;
            o_lfsr[6] <= o_lfsr[5];
            o_lfsr[7] <= o_lfsr[6];
            o_lfsr[8] <= o_lfsr[7];
            o_lfsr[9] <= o_lfsr[8];
            o_lfsr[10] <= o_lfsr[9];
            o_lfsr[11] <= o_lfsr[10];
            o_lfsr[12] <= o_lfsr[11];
            o_lfsr[13] <= o_lfsr[12];
            o_lfsr[14] <= o_lfsr[13];
            o_lfsr[15] <= o_lfsr[14];

            // valid
            o_valid <= i_valid;
        end
        else
        begin
            o_valid <= 1'b0;
        end
    end
    assign feedback = o_lfsr[15];

endmodule
```

Imagen 2.3 Diseño del generador

checker.v

Diseño del módulo checker, este módulo consiste en una máquina de dos estados (LOCK y UNLOCK) que pivota entre ambos según thresholds definidos.
Recibe el dato producido por el generador, en primera instancia toma ese dato como referencia para calcular el próximo. Una vez ocurra un MATCH (dato recibido == dato predicho) entonces utilizará su propia lógica interna para calcular los siguientes valores de la secuencia.

Se definen *LOCK_THR* y *UNLOCK_THR* los cuales indican la cantidad de MATCHES y MISMATCHES respectivamente para pasar de un estado a otro.

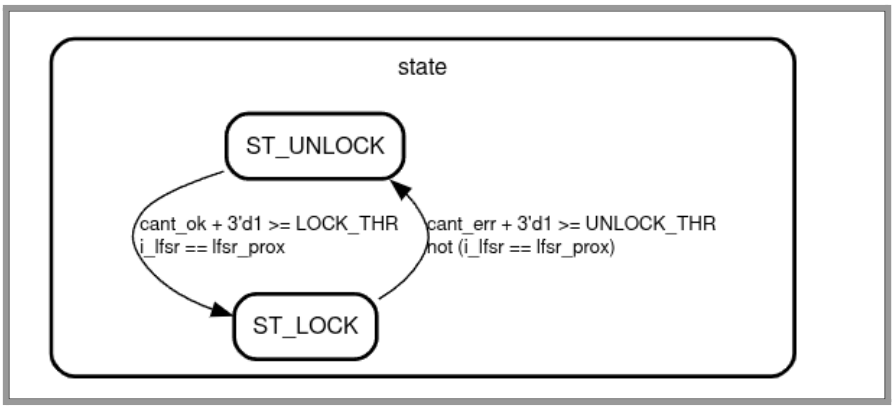


Imagen 2.4 Diagrama de estados de checker

Comienza en estado UNLOCK, hasta que recibe *LOCK_THR* MATCHES consecutivos, en ese momento pasa a estado LOCK, estado el cual es deseable que se mantenga para siempre, caso contrario significa que la comunicación no está siendo fiable. En estado LOCK soporta *UNLOCK_THR* MISMATCHES consecutivos antes de pasar de nuevo a estado UNLOCK.

Este comportamiento esperado del diseño del checker es poder comenzar a funcionar en cualquier momento independientemente de cuando lo hizo el generador, y llegar a lockearse. También tolerar rafagas de fallas en la comunicación de hasta UNLOCK_THR fallas. Si se llegase a deslockear, poder ser lo suficientemente robusto como para volver a lockearse nuevamente cuando la comunicación se recomponga. Esta lógica permite ese comportamiento de manera óptima.

```
module lfsr_checker #(
    parameter integer LOCK_THR   = 2,
    parameter integer UNLOCK_THR = 5
)(
    //----> Inputs
    input  wire        clock,
    input  wire        i_rst,
    input wire       i_enable,

    input  wire        i_valid,
    input  wire [15:0] i_lfsr,

    //----> Outputs
    output reg                  o_lock,
    output wire [15:0]         o_checker
);
```

Imagen 2.5 Entradas y salidas del checker

Parámetros

Parámetro	Comportamiento
LOCK_THR	Cantidad de MATCHES consecutivos para lockear
UNLOCK_THR	Cantidad de MISMATCHES consecutivos para deslockear

Entradas

Entrada	Comportamiento
clock	Señal de clock
i_rst	Reset asíncrono
i_enable	Habilita el checker
i_valid	Señal de valid que viene de la salida del generador
i_lfsr	*Dato que viene de la salida del generador*

Salidas

Salida	Comportamiento
o_lock	Variable que indica el estado del sistema LOCK/UNLOCK
o_checker	Indica valor predicho

```
if (i_valid && i_enable) begin

    case (state)

        //-------------------------------------------------------------
        // UNLOCK: Espera LOCK_THR matches consecutivos.
        ST_UNLOCK: begin
            if (i_lfsr == lfsr_prox) begin // si matchea la palabra con la predicción

                lfsr_prox <= lfsr_next(lfsr_prox); // recalculamos el siguiente valor esperado

                if (cant_ok + 3'd1 >= LOCK_THR) begin
                    // Suficientes aciertos consecutivos → LOCK
                    state    <= ST_LOCK;
                    cant_ok  <= 3'd0;
                    cant_err <= 3'd0;
                    o_lock <= 1'b1;
                end else begin
                    cant_ok  <= cant_ok + 3'd1;
                end
            end

             else begin // no matchea la palabra con la predicción
                lfsr_prox <= lfsr_next(i_lfsr); // i_lfsr pasa a ser el nuevo ancla
                cant_ok   <= 3'd0;
            end
        end


        //-------------------------------------------------------------
        // LOCK: Sincronizado con el generador.
        // Sale a UNLOCK tras UNLOCK_THR fallos consecutivos.
        ST_LOCK: begin
            lfsr_prox <= lfsr_next(lfsr_prox); //calculo la siguiente palabra esperada pero con el valor anterior de

            if (i_lfsr == lfsr_prox) begin // si matchea la palabra con la predicción
                cant_err <= 3'd0;
            end

            else    // Fallo detectado
            begin

                cant_err <= cant_err + 3'd1;
                //o_error  <= 1'b1; // Señala error durante este ciclo

                if (cant_err + 3'd1 >= UNLOCK_THR)   // Demasiados fallos consecutivos → UNLOCK
                begin
                    state    <= ST_UNLOCK;
                    cant_err <= 3'd0;
                    cant_ok  <= 3'd0;
                    seeded   <= 1'b0;   // necesita nueva seed
                    o_lock <= 1'b0;
                end
            end
        end

    endcase
```

Imagen 2.6 Diseño máquina de estados checker

top_lfsr.v

Este módulo es el encargado de realizar la conexión entre ambos bloques.

```
    // ============================================================================
    // INSTANCIA DEL GENERADOR LFSR
    // ============================================================================
    lfsr #(
        .SEED(SEED)
    ) u_generator (
        // Inputs
        .clock        (clock),
        .i_rst        (i_rst),
        .i_soft_reset (i_soft_reset),
        .i_enable     (i_gen_enable),
        .i_valid      (i_gen_valid),
        .i_seed       (i_seed),

        // Outputs (Se conectan directamente a las salidas del top)
        .o_lfsr         (o_lfsr),
        .o_valid      (o_gen_valid)
    );


    // ============================================================================
    // INSTANCIA DEL CHECKER
    // ============================================================================

    lfsr_checker #(
        .LOCK_THR   (LOCK_THR),
        .UNLOCK_THR (UNLOCK_THR)
    ) u_checker (
        // Inputs
        .clock        (clock),
        .i_rst        (i_rst_checker),
        .i_enable     (i_checker_enable),

        // Entradas de datos provenientes del generador
        .i_valid      (i_checker_valid), // Conectado a la salida valid del generador
        .i_lfsr       (i_checker_data),  // Conectado a la salida de datos del generador

        // Outputs
        .o_checker     (o_checker), // Salida de datos del checker hacia el exterior
        .o_lock       (o_lock) // Salida de lock hacia el exterior
    );
endmodule
```

Imagen 2.7 Instancia de módulos en top

Testing

Para realizar el testing se definio un testbench (***top_LFSR_tb.sv***) en el cual instancia el top y se definen diferentes tasks utilizadas para los distintos tipos de resets, seteo de semilla para soft reset, monitor de una determinada cantidad de señales *valid*, inyección de errores controlada sobre una cola intermediaria, y un proceso que controla constantemente el estado de lockeo del checker.

Se definió también un archivo clave para las pruebas llamado ***queue.sv****,* donde se describe una cola FIFO actuará como intermediaria entre el generador y el checker en el testeo, esto permite agregar delays random entre la salida del generador y el checker, así como también poder corromper datos a demanda. Todo esto para poder simular una situación más real donde se podrían encontrar estos módulos, a km de distancia y con un canal que pueda ser más caótico.

```
reg        inject_error = 0;


// Declaración de Queue
// El símbolo [$] indica que es de tamaño dinámico
reg [15:0] delay_queue [$];
// --- Señales que irán conectadas a los inputs del Checker ---
reg [15:0] queue_data_out;
reg        queue_valid_out;

always @(posedge clock) begin
    if (i_rst) begin
        delay_queue.delete(); // Vaciamos la cola al resetear
        queue_valid_out <= 1'b0;
    end else begin

        // ------------------------------------------------------
        // 1. ESCRITURA EN LA COLA (PUSH) - Lado del Generador
        // ------------------------------------------------------
        if (o_gen_valid) begin //gracias a este if tendremos guardados solo datos validos en la cola
            // Si además queremos inyectar un error, lo hacemos aca
            if (inject_error)
                delay_queue.push_back(o_lfsr ^ 16'h0001); // Inyectar error
            else
                delay_queue.push_back(o_lfsr); // Dato sano
        end



        // ------------------------------------------------------
        // 2. LECTURA DE LA COLA (POP) - Lado del Checker
        // ------------------------------------------------------

        // delay_queue.size() nos dice cuántos elementos hay guardados.
        // $urandom_range(1, 100) <= 30) es un "sorteo" para decidir si sacamos un dato o no (30% de p
        // de cantidad de ciclos aleatorios que le tome ganar el sorteo.
        if (i_checker_enable && delay_queue.size() > 0 && ($urandom_range(1, 100) <= 30)) begin
            // Hay datos y ganamos el sorteo
            queue_data_out  <= delay_queue.pop_front(); // Sacamos el más viejo
            queue_valid_out <= 1'b1;                    // Le avisamos al checker que hay dato
        end else begin
            // La cola está vacía, o simplemente "decidimos" retener el dato este ciclo
            queue_valid_out <= 1'b0;
        end
    end
end
```

Imagen 2.8 queue.sv

Al tener esta cola intermediaria, en los archivos de testeo las salidas del generador van directamente a la cola y las salidas de la cola pasan a ser las entradas del checker.
También vemos como con la flag *inject_error* se puede corromper un dato a demanda, aplicando una operación XOR.

```
// Inyector de Errores (Se usa con fork/join_any)
task error_injector(input integer num_good, input integer num_bad);
    begin
        // Bucle infinito: intercala N datos sanos con M datos corruptos
        while(1) begin
            repeat(num_good) begin
                #1;
                inject_error = 0;
                @(posedge clock);
                while(!o_gen_valid)
                    @(posedge clock); // Esperamos a que salga un dato real
            end
            repeat(num_bad) begin
                #1;
                inject_error = 1;
                @(posedge clock);
                while(!o_gen_valid)
                    @(posedge clock);
            end
        end
    end
endtask
```

Imagen 2.9 error_injector task

Esta task se encarga de cambiar la flag *inject_error* de la queue segun se desee.

Se utiliza este proceso paralelo para el monitoreo constante de la salida *o_lock*

```
// 2. Proceso de Monitoreo constante de o_lock
initial begin
    prev_lock = 0;
    forever begin
        @(posedge clock);
        if (o_lock !== prev_lock) begin

            if (o_lock == 1'b1)
            begin
                $display("[%0t ns] LOCK: El estado de lock cambió a %b", $time, o_lock);
                lock_cnt++;
            end
            else
            begin
                $display("[%0t ns] UNLOCK: El estado de lock cambió a %b", $time, o_lock);
                unlock_cnt++;
            end
            prev_lock = o_lock;
        end
    end
end
```

Imagen 2.10 Proceso monitoreo de estado de lockeo

También se definió un archivo llamado *valid_generator.sv* el cual cumple la función de generar la señal de valid para los tests, permitiendo generarla de forma caótica para poder probar todo tipo de escenarios.

```
always @(posedge clock) begin
    if (i_rst) begin

        valid_signal <= 1'b0;
        valid_cicle_counter <= 0;

        if (valid_random_cycles == 1)
            begin
                valid_wait_cycles <= $urandom_range(MIN_WAIT, MAX_WAIT);
            end
        else begin
            valid_wait_cycles <= DEFAULT_WAIT_CYCLES;
        end

    end


    else if (i_soft_reset) begin
        valid_signal <= 1'b0;
        valid_cicle_counter <= 0;

        if (valid_random_cycles == 1)
            begin
                valid_wait_cycles <= $urandom_range(MIN_WAIT, MAX_WAIT);
            end
        else begin
            valid_wait_cycles <= DEFAULT_WAIT_CYCLES;
        end

    end

    else begin
        // si el contador alcanzo el limite aleatorio que estábamos esperando
        if (valid_cicle_counter == valid_wait_cycles)
        begin

            if(valid_signal == 1'b1) begin
                valid_signal <= 1'b0; // 1. Apagamos el pulso
            end
            else begin
                valid_signal <= 1'b1; // 2. Mandamos el pulso
            end
            valid_cicle_counter <= 0;           // 2. Reiniciamos el contador

            //seteamos el limite aleatorio para el próximo pulso
            if (valid_random_cycles == 1)
            begin
                valid_wait_cycles <= $urandom_range(MIN_WAIT, MAX_WAIT);
            end
            else begin
                valid_wait_cycles <= DEFAULT_WAIT_CYCLES;
            end
        end
        else begin
            valid_cicle_counter <= valid_cicle_counter + 1;
        end
    end
end
```

Imagen 2.11 valid_generator.sv

Luego de crear estas herramientas se definieron ahora si los tests modulares a incluir con `ifdef según se desee.

Todos los tests se realizaron en systemverilog por mayor comodidad en herramientas de testeo, se priorizo utilizar tiempos o valores random cada vez que fue posible y alrededor de 100 iteraciones por test para probar una gran cantidad de casos con variabilidad.

Resumen de tests

A continuación se encuentra un resumen de los primeros tests, y un desarrollo más exhaustivo de los que se considera que lo ameritan en el informe. En total se realizaron siete tests modulares.

Test	Qué verifica	Técnica utilizada
TEST1	Comportamiento del generador esperado	Se inicia el generador, se deja funcionar un tiempo random y se compara su salida constantemente con un modelo matemático de comportamiento esperado, definido para el testing.
TEST2	Periodicidad del generador	Se parte del conocimiento que la secuencia deberia repetirse luego de 65535 taps. Se genera un valor, se guarda y se espera 65535 datos. Se compara el nuevo dato con el guardado anteriormente y deben ser iguales.
TEST3	Verifica que el checker funcione correctamente en condiciones normales.	Se habilita el generador y luego de un tiempo random el checker. Se comparan los valores generados con los esperados matemáticamente. Se espera que el total de matches correctos sea igual a n-1 por cada iteración, ya que el primero debería ser MISMATCH porque es el dato que se toma como ancla.
TEST4	Test de Fronteras (Thresholds)	*Se desarrolla mas adelante.*
TEST5	Test de LOCK/UNLOCK (PING-PONG)	*Se desarrolla mas adelante.*
TEST6	Resiliencia ante Resets randoms.	*Se desarrolla mas adelante.*
TEST7	Cambio Dinámico de Semilla.	*Se desarrolla mas adelante.*

Test 4 Test de Fronteras (Thresholds)

Verifica que el checker respete los parámetros de LOCK y UNLOCK mediante la inyección paralela de errores. Se divide en dos partes TEST A y TEST B.

TEST A : Nunca lockee (Inyectamos menos datos buenos que el LOCK_THR).
Esto hace que nunca lockee, debería esperar verse 2 MISMATCH seguidos, (dato corrupto y el que sigue con ancla corrupta) ya que al no machear el checker sigue usando la entrada para predecir el siguiente valor.

```
// ------------------------------------------------------------------------
// TEST A: NUNCA LOCKEE  (inyectamos menos datos buenos que el LOCK_THR)
// ------------------------------------------------------------------------

reset();
reset_checker();
i_gen_enable = ENABLE;

// Abrimos hilos con inyector de errores
fork
    // Hilo 1:  inyector de errores
    //metemos LOCK_THR-1 datos buenos para que no llegue a lockear
    error_injector(LOCK_THR-1, 1);


    // Hilo 2: El monitor principal
    begin
        #($urandom_range(50,500) * 1ns); //dejamos que se generen algunos valids antes de habilitar el checker
        @(posedge clock);
        i_checker_enable = ENABLE;
        monitor_valids(30, "TEST4-A NUNCA LOCK"); //testeamos 20 datos, deberia ver 2 MISMATCH seguidos y nunca lockear
        // vemos si se lockeo o no
        if (o_lock !== 1'b0) begin
            $error("[%0t ns] FALLO: El checker se lockeó y no debia.", $time);
            $finish;
        end else begin
            $display("[%0t ns] EXITO: El checker soportó la prueba Nunca Lock.", $time);
        end
    end
join_any

disable fork;
inject_error = 0;
i_checker_enable = DISABLE;
i_gen_enable = DISABLE;
```

Imagen 2.12 Test 4 A

Se utiliza fork para llamar paralelamente a la inyección de errores.

```
--------- TEST4-A NUNCA LOCK ---------
[7342425000 ns] Valid #1  | Generador: 3452 | Recibido: ffd3 | Checker: 0000 | MISMATCH | Matches totales: 0  |  Lock: 0
[7342445000 ns] Valid #2  | Generador: d148 | Recibido: ff8b | Checker: ff8b | MATCH    | Matches totales: 1  |  Lock: 0
[7342465000 ns] Valid #3  | Generador: a2bd | Recibido: ff3b | Checker: ff3b | MATCH    | Matches totales: 2  |  Lock: 0
[7342555000 ns] Valid #4  | Generador: 1571 | Recibido: fe5b | Checker: fe5b | MATCH    | Matches totales: 3  |  Lock: 0
[7342615000 ns] Valid #5  | Generador: 55c4 | Recibido: fc9a | Checker: fc9b | MISMATCH | Matches totales: 3  |  Lock: 0
[7342625000 ns] Valid #6  | Generador: 55c4 | Recibido: f91b | Checker: f919 | MISMATCH | Matches totales: 3  |  Lock: 0
[7342645000 ns] Valid #7  | Generador: ab88 | Recibido: f21b | Checker: f21b | MATCH    | Matches totales: 4  |  Lock: 0
[7342715000 ns] Valid #8  | Generador: e692 | Recibido: e41b | Checker: e41b | MATCH    | Matches totales: 5  |  Lock: 0
[7342835000 ns] Valid #9  | Generador: a2b5 | Recibido: c81b | Checker: c81b | MATCH    | Matches totales: 6  |  Lock: 0
[7342845000 ns] Valid #10 | Generador: a2b5 | Recibido: 901a | Checker: 901b | MISMATCH | Matches totales: 6  |  Lock: 0
[7342885000 ns] Valid #11 | Generador: a2b5 | Recibido: 201b | Checker: 2019 | MISMATCH | Matches totales: 6  |  Lock: 0
[7342935000 ns] Valid #12 | Generador: 2a62 | Recibido: 4036 | Checker: 4036 | MATCH    | Matches totales: 7  |  Lock: 0
[7343055000 ns] Valid #13 | Generador: 9a65 | Recibido: 806c | Checker: 806c | MATCH    | Matches totales: 8  |  Lock: 0
[7343085000 ns] Valid #14 | Generador: 9a65 | Recibido: 00f5 | Checker: 00f5 | MATCH    | Matches totales: 9  |  Lock: 0
[7343105000 ns] Valid #15 | Generador: 69ce | Recibido: 01eb | Checker: 01ea | MISMATCH | Matches totales: 9  |  Lock: 0
[7343115000 ns] Valid #16 | Generador: d39c | Recibido: 03d4 | Checker: 03d6 | MISMATCH | Matches totales: 9  |  Lock: 0
[7343125000 ns] Valid #17 | Generador: a715 | Recibido: 07a8 | Checker: 07a8 | MATCH    | Matches totales: 10  |  Lock: 0
[7343135000 ns] Valid #18 | Generador: 4e07 | Recibido: 0f50 | Checker: 0f50 | MATCH    | Matches totales: 11  |  Lock: 0
[7343175000 ns] Valid #19 | Generador: 3831 | Recibido: 1ea0 | Checker: 1ea0 | MATCH    | Matches totales: 12  |  Lock: 0
[7343185000 ns] Valid #20 | Generador: 3831 | Recibido: 3d41 | Checker: 3d40 | MISMATCH | Matches totales: 12  |  Lock: 0
[7343225000 ns] Valid #21 | Generador: c1a5 | Recibido: 7a80 | Checker: 7a82 | MISMATCH | Matches totales: 12  |  Lock: 0
[7343255000 ns] Valid #22 | Generador: 0dc6 | Recibido: f500 | Checker: f500 | MATCH    | Matches totales: 13  |  Lock: 0
[7343265000 ns] Valid #23 | Generador: 1b8c | Recibido: ea2d | Checker: ea2d | MATCH    | Matches totales: 14  |  Lock: 0
[7343275000 ns] Valid #24 | Generador: 1b8c | Recibido: d477 | Checker: d477 | MATCH    | Matches totales: 15  |  Lock: 0
[7343325000 ns] Valid #25 | Generador: dc60 | Recibido: a8c2 | Checker: a8c3 | MISMATCH | Matches totales: 15  |  Lock: 0
[7343375000 ns] Valid #26 | Generador: 8fcf | Recibido: 51ab | Checker: 51a9 | MISMATCH | Matches totales: 15  |  Lock: 0
[7343395000 ns] Valid #27 | Generador: 1fb3 | Recibido: a356 | Checker: a356 | MATCH    | Matches totales: 16  |  Lock: 0
[7343525000 ns] Valid #28 | Generador: d82b | Recibido: 4681 | Checker: 4681 | MATCH    | Matches totales: 17  |  Lock: 0
[7343545000 ns] Valid #29 | Generador: d82b | Recibido: 8d02 | Checker: 8d02 | MATCH    | Matches totales: 18  |  Lock: 0
[7343585000 ns] Valid #30 | Generador: 8341 | Recibido: 1a28 | Checker: 1a29 | MISMATCH | Matches totales: 18  |  Lock: 0
[7343595000 ns] EXITO: El checker soportó la prueba Nunca Lock.
```

Imagen 2.13 Salida por consola Test 4 A

Se puede ver en la consola como nunca llega a cambiar a estado LOCK (Lock siempre es 0) ya que se registran como máximo 3 matches consecutivos, cuando deberian ser 5 para cambiar de estado segun los thresholds definidos.

TEST B : Nunca se desbloquee (Inyectamos menos datos malos que el UNLOCK_THR)

Se hace uso de la task definida en *top_LFSR*_tb.sv la cual cambia la flag *inject_error* de la queue.

```
#500ns;
// ------------------------------------------------------------------------
// TEST B : Nunca se desbloquee (Inyectamos menos datos malos que el UNLOCK_THR)
// ------------------------------------------------------------------------
reset();
reset_checker();
i_gen_enable = ENABLE;
i_checker_enable = ENABLE;

// Primero dejamos que se lockee
// metemos LOCK_THR+1 datos buenos para que lockee
monitor_valids(LOCK_THR +1 , "TEST4-B NUNCA UNLOCK (Fase de anclaje)"); //LOCK_THR+1 (el primero siempre es MISMATCH)

// Ahora que está en lockeado, empezamos a meter errores
fork
    //metemos UNLOCK_THR-1 datos corruptos para que no llegue a deslockearse.
    error_injector(5, UNLOCK_THR-1); // 5 buenos, 2 Corruptos
    begin
        monitor_valids(30, "TEST4-B NUNCA UNLOCK (Fase de inyección de errores)");
        if (o_lock !== 1'b1 ) begin
            $error("[%0t ns] FALLO: El checker perdió el lock y no debía.", $time);
            $finish;
        end else begin //else if porque o_lock puede ser X y seguir funcionando, pero no deberia serlo
            $display("[%0t ns] EXITO: El checker soportó la prueba Nunca Lock.", $time);
        end
    end
join_any

disable fork;
inject_error = 0;
i_checker_enable = DISABLE;
i_gen_enable = DISABLE;
```

Imagen 2.14 Test 4 B

Vemos como utilizamos fork nuevamente para inyectar errores, y en este test tambien tenemos una fase de anclaje en la cual esperamos a *o_lock == 1* lo que indica que esta lockeado y debe mantenerse en ese estado.

```
--------- TEST4-B NUNCA UNLOCK (Fase de anclaje) ---------
[7381415000 ns] Valid #1 | Generador: ff8b | Recibido: ffd3 | Checker: 0000 | MISMATCH | Matches totales: 0  |  Lock: 0
[7381435000 ns] Valid #2 | Generador: fe5b | Recibido: ff8b | Checker: ff8b | MATCH    | Matches totales: 1  |  Lock: 0
[7381455000 ns] Valid #3 | Generador: f91b | Recibido: ff3b | Checker: ff3b | MATCH    | Matches totales: 2  |  Lock: 0
[7381485000 ns] Valid #4 | Generador: c81b | Recibido: fe5b | Checker: fe5b | MATCH    | Matches totales: 3  |  Lock: 0
[7381515000 ns] Valid #5 | Generador: 201b | Recibido: fc9b | Checker: fc9b | MATCH    | Matches totales: 4  |  Lock: 0
[7381565000 ns] Valid #6 | Generador: 201b | Recibido: f91b | Checker: f91b | MATCH    | Matches totales: 5  |  Lock: 0

--------- TEST4-B NUNCA UNLOCK (Fase de inyeccn de errores) ---------
[7381575000 ns] LOCK: El estado de lock cambió a 1
[7381595000 ns] Valid #1 | Generador: 00f5 | Recibido: f21b | Checker: f21b | MATCH    | Matches totales: 1  |  Lock: 1
[7381615000 ns] Valid #2 | Generador: 03d4 | Recibido: e41b | Checker: e41b | MATCH    | Matches totales: 2  |  Lock: 1
[7381635000 ns] Valid #3 | Generador: 03d4 | Recibido: c81b | Checker: c81b | MATCH    | Matches totales: 3  |  Lock: 1
[7381655000 ns] Valid #4 | Generador: 03d4 | Recibido: 901b | Checker: 901b | MATCH    | Matches totales: 4  |  Lock: 1
[7381715000 ns] Valid #5 | Generador: 0f50 | Recibido: 201b | Checker: 201b | MATCH    | Matches totales: 5  |  Lock: 1
[7381755000 ns] Valid #6 | Generador: 0f50 | Recibido: 4036 | Checker: 4036 | MATCH    | Matches totales: 6  |  Lock: 1
[7381835000 ns] Valid #7 | Generador: f500 | Recibido: 806c | Checker: 806c | MATCH    | Matches totales: 7  |  Lock: 1
[7381935000 ns] Valid #8 | Generador: f500 | Recibido: 00f5 | Checker: 00f5 | MATCH    | Matches totales: 8  |  Lock: 1
[7382005000 ns] Valid #9 | Generador: 8d02 | Recibido: 01ea | Checker: 01ea | MATCH    | Matches totales: 9  |  Lock: 1
[7382045000 ns] Valid #10 | Generador: 3452 | Recibido: 03d4 | Checker: 03d4 | MATCH    | Matches totales: 10  |  Lock: 1
[7382075000 ns] Valid #11 | Generador: a2bd | Recibido: 07a8 | Checker: 07a8 | MATCH    | Matches totales: 11  |  Lock: 1
[7382085000 ns] Valid #12 | Generador: a2bd | Recibido: 0f51 | Checker: 0f50 | MISMATCH | Matches totales: 11  |  Lock: 1
[7382095000 ns] Valid #13 | Generador: a2bd | Recibido: 1ea1 | Checker: 1ea0 | MISMATCH | Matches totales: 11  |  Lock: 1
[7382115000 ns] Valid #14 | Generador: a2bd | Recibido: 3d40 | Checker: 3d40 | MATCH    | Matches totales: 12  |  Lock: 1
[7382155000 ns] Valid #15 | Generador: 4557 | Recibido: 7a80 | Checker: 7a80 | MATCH    | Matches totales: 13  |  Lock: 1
[7382165000 ns] Valid #16 | Generador: 8aae | Recibido: f500 | Checker: f500 | MATCH    | Matches totales: 14  |  Lock: 1
[7382215000 ns] Valid #17 | Generador: 573d | Recibido: ea2d | Checker: ea2d | MATCH    | Matches totales: 15  |  Lock: 1
[7382255000 ns] Valid #18 | Generador: 7349 | Recibido: d477 | Checker: d477 | MATCH    | Matches totales: 16  |  Lock: 1
[7382265000 ns] Valid #19 | Generador: 7349 | Recibido: a8c2 | Checker: a8c3 | MISMATCH | Matches totales: 16  |  Lock: 1
[7382275000 ns] Valid #20 | Generador: 7349 | Recibido: 51aa | Checker: 51ab | MISMATCH | Matches totales: 16  |  Lock: 1
[7382315000 ns] Valid #21 | Generador: cd09 | Recibido: a356 | Checker: a356 | MATCH    | Matches totales: 17  |  Lock: 1
[7382325000 ns] Valid #22 | Generador: 9a3f | Recibido: 4681 | Checker: 4681 | MATCH    | Matches totales: 18  |  Lock: 1
[7382355000 ns] Valid #23 | Generador: 9a3f | Recibido: 8d02 | Checker: 8d02 | MATCH    | Matches totales: 19  |  Lock: 1
[7382385000 ns] Valid #24 | Generador: d14c | Recibido: 1a29 | Checker: 1a29 | MATCH    | Matches totales: 20  |  Lock: 1
[7382415000 ns] Valid #25 | Generador: a2b5 | Recibido: 3452 | Checker: 3452 | MATCH    | Matches totales: 21  |  Lock: 1
[7382445000 ns] Valid #26 | Generador: a2b5 | Recibido: 68a5 | Checker: 68a4 | MISMATCH | Matches totales: 21  |  Lock: 1
[7382465000 ns] Valid #27 | Generador: 4547 | Recibido: d149 | Checker: d148 | MISMATCH | Matches totales: 21  |  Lock: 1
[7382475000 ns] Valid #28 | Generador: 8a8e | Recibido: a2bd | Checker: a2bd | MATCH    | Matches totales: 22  |  Lock: 1
[7382535000 ns] Valid #29 | Generador: a67a | Recibido: 4557 | Checker: 4557 | MATCH    | Matches totales: 23  |  Lock: 1
[7382565000 ns] Valid #30 | Generador: 99b2 | Recibido: 8aae | Checker: 8aae | MATCH    | Matches totales: 24  |  Lock: 1
[7382575000 ns] EXITO: El checker soportó la prueba Nunca Lock.
```

Imagen 2.15 Salida por consola Test 4 B

Vemos como a la salida por consola, tenemos primero la fase de anclade donde como es esperado el estado de lock pase de 0 a 1, es decir se lockee.

Luego no sale nunca del estado de LOCK ya que la cantidad de datos corruptos recibidos (MISMATCHES) son menores al threshold definido de 3.

Test 5 Test de LOCK/UNLOCK (PING-PONG)

Verifica que el checker cambie constantemente de estado LOCK a UNLOCK y viceversa, mediante la inyección de errores controlados

Se utiliza la tarea *error_injector* nuevamente en un hilo definido con un fork, para forzar el cambio de estado constantemente.

```
[13029445000 ns] UNLOCK: El estado de lock cambió a 0
[13029525000 ns] Valid #145 | Generador: 8801 | Recibido: d519 | Checker: d519 | MATCH    | Matches totales: 96  |  Lock: 0
[13029535000 ns] Valid #146 | Generador: 8801 | Recibido: aa1f | Checker: aa1f | MATCH    | Matches totales: 97  |  Lock: 0
[13029555000 ns] Valid #147 | Generador: 8801 | Recibido: 5413 | Checker: 5413 | MATCH    | Matches totales: 98  |  Lock: 0
[13029565000 ns] Valid #148 | Generador: 102f | Recibido: a826 | Checker: a826 | MATCH    | Matches totales: 99  |  Lock: 0
[13029585000 ns] Valid #149 | Generador: 40bc | Recibido: 5061 | Checker: 5061 | MATCH    | Matches totales: 100  |  Lock: 0
[13029595000 ns] LOCK: El estado de lock cambió a 1
[13029605000 ns] Valid #150 | Generador: 02dd | Recibido: a0c2 | Checker: a0c2 | MATCH    | Matches totales: 101  |  Lock: 1
[13029625000 ns] Valid #151 | Generador: 0b74 | Recibido: 41a8 | Checker: 41a9 | MISMATCH | Matches totales: 101  |  Lock: 1
[13029685000 ns] Valid #152 | Generador: 0b74 | Recibido: 8353 | Checker: 8352 | MISMATCH | Matches totales: 101  |  Lock: 1
[13029715000 ns] Valid #153 | Generador: 0b74 | Recibido: 0688 | Checker: 0689 | MISMATCH | Matches totales: 101  |  Lock: 1
[13029725000 ns] Valid #154 | Generador: 0b74 | Recibido: 0d12 | Checker: 0d12 | MATCH    | Matches totales: 102  |  Lock: 0
[13029725000 ns] UNLOCK: El estado de lock cambió a 0
[13029735000 ns] Valid #155 | Generador: 0b74 | Recibido: 1a24 | Checker: 1a24 | MATCH    | Matches totales: 103  |  Lock: 0
[13029775000 ns] Valid #156 | Generador: b740 | Recibido: 3448 | Checker: 3448 | MATCH    | Matches totales: 104  |  Lock: 0
[13029805000 ns] Valid #157 | Generador: 6ead | Recibido: 6890 | Checker: 6890 | MATCH    | Matches totales: 105  |  Lock: 0
[13029815000 ns] Valid #158 | Generador: 6ead | Recibido: d120 | Checker: d120 | MATCH    | Matches totales: 106  |  Lock: 0
[13029825000 ns] LOCK: El estado de lock cambió a 1
[13029845000 ns] Valid #159 | Generador: ba99 | Recibido: a26d | Checker: a26d | MATCH    | Matches totales: 107  |  Lock: 1
[13029855000 ns] Valid #160 | Generador: ba99 | Recibido: 44f6 | Checker: 44f7 | MISMATCH | Matches totales: 107  |  Lock: 1
[13029865000 ns] Valid #161 | Generador: ba99 | Recibido: 89ef | Checker: 89ee | MISMATCH | Matches totales: 107  |  Lock: 1
[13029875000 ns] Valid #162 | Generador: 751f | Recibido: 13f0 | Checker: 13f1 | MISMATCH | Matches totales: 107  |  Lock: 1
[13029885000 ns] Valid #163 | Generador: ea3e | Recibido: 27e2 | Checker: 27e2 | MATCH    | Matches totales: 108  |  Lock: 0
[13029885000 ns] UNLOCK: El estado de lock cambió a 0
[13029945000 ns] Valid #164 | Generador: a88f | Recibido: 4fc4 | Checker: 4fc4 | MATCH    | Matches totales: 109  |  Lock: 0
[13030015000 ns] Valid #165 | Generador: 13a9 | Recibido: 9f88 | Checker: 9f88 | MATCH    | Matches totales: 110  |  Lock: 0
[13030035000 ns] Valid #166 | Generador: 4ea4 | Recibido: 3f3d | Checker: 3f3d | MATCH    | Matches totales: 111  |  Lock: 0
[13030085000 ns] Valid #167 | Generador: 4ea4 | Recibido: 7e7a | Checker: 7e7a | MATCH    | Matches totales: 112  |  Lock: 0
[13030095000 ns] LOCK: El estado de lock cambió a 1
[13030165000 ns] Valid #168 | Generador: 3abd | Recibido: fcf4 | Checker: fcf4 | MATCH    | Matches totales: 113  |  Lock: 1
[13030225000 ns] Valid #169 | Generador: 757a | Recibido: f9c4 | Checker: f9c5 | MISMATCH | Matches totales: 113  |  Lock: 1
[13030305000 ns] Valid #170 | Generador: 5da1 | Recibido: f3a6 | Checker: f3a7 | MISMATCH | Matches totales: 113  |  Lock: 1
[13030335000 ns] Valid #171 | Generador: ed52 | Recibido: e762 | Checker: e763 | MISMATCH | Matches totales: 113  |  Lock: 1
[13030345000 ns] UNLOCK: El estado de lock cambió a 0
[13030385000 ns] Valid #172 | Generador: b53f | Recibido: ceeb | Checker: ceeb | MATCH    | Matches totales: 114  |  Lock: 0
[13030445000 ns] Valid #173 | Generador: b53f | Recibido: 9dfb | Checker: 9dfb | MATCH    | Matches totales: 115  |  Lock: 0
[13030485000 ns] Valid #174 | Generador: a961 | Recibido: 3bdb | Checker: 3bdb | MATCH    | Matches totales: 116  |  Lock: 0
[13030535000 ns] Valid #175 | Generador: 2e69 | Recibido: 77b6 | Checker: 77b6 | MATCH    | Matches totales: 117  |  Lock: 0
[13030555000 ns] Valid #176 | Generador: 5cd2 | Recibido: ef6c | Checker: ef6c | MATCH    | Matches totales: 118  |  Lock: 0
[13030565000 ns] LOCK: El estado de lock cambió a 1
[13030575000 ns] Valid #177 | Generador: 5cd2 | Recibido: def5 | Checker: def5 | MATCH    | Matches totales: 119  |  Lock: 1
[13030595000 ns] Valid #178 | Generador: 5cd2 | Recibido: bdc6 | Checker: bdc7 | MISMATCH | Matches totales: 119  |  Lock: 1
[13030605000 ns] Valid #179 | Generador: 5cd2 | Recibido: 7ba2 | Checker: 7ba3 | MISMATCH | Matches totales: 119  |  Lock: 1
[13030625000 ns] Valid #180 | Generador: 5cd2 | Recibido: f747 | Checker: f746 | MISMATCH | Matches totales: 119  |  Lock: 1
[13030635000 ns] UNLOCK: El estado de lock cambió a 0
[13030636000 ns] EXITO: El checker hizo ping-pong perfectamente (40 transiciones).
```

Imagen 2.16 Salida por consola Test 5

Vemos como a la salida tenemos un cambio continuamente entre estados. Indicados por el proceso que está monitoreando continuamente la salida *o_lock*.

Test 6 Resiliencia ante Resets randoms

Verifica que el sistema pueda recuperarse y volver a lockearse tras un reset repentino en pleno funcionamiento.

Se resetea solo el checker en mitad de funcionamiento del generador y se verifica que al arrancar de nuevo pueda pasar del estado UNLOCK a LOCK.

```
    $display("[%0t ns] ---> INYECTANDO RESET", $time);

    fork
        begin
            // Hilo A: resetea checker (que mantiene el reset por 1us-249us y luego lo baja)
            reset_checker();
        end
        begin
            // Hilo B: Le damos 1 nanosegundo al simulador para que procese la acción de reset y verifica
            #1;
            if (o_lock !== 1'b0) begin
                $error("[%0t ns] FALLO CRITICO: El reset no bajó la señal de Lock.", $time);
                $finish;
            end
            $display("[%0t ns] Deslockeado con éxito. El checker reaccionó al reset.", $time);
        end
    join // Join porque queremos que espere a reset_checker()

    // ------------------------------------------------------------
    // 3. RECUPERACIÓN
    // ------------------------------------------------------------
    $display("[%0t ns] Reset liberado. Esperando que el sistema se recupere por sí solo...", $time);

    // Esperamos que vuelva a lockear, pero con un timeout de seguridad
    fork : hilo_timeout
        begin
            wait(o_lock == 1'b1);
            $display("[%0t ns] EXITO: El sistema sobrevivió al reset y volvió a lockearse.", $time);
        end
        begin
            #50000ns; // Timeout
            $error("[%0t ns] FALLO: El sistema nunca se recuperó del reset asesino.", $time);
            $finish;
        end
    join_any

    // Matamos el hilo del timeout apenas logre lockear
    disable hilo_timeout;
```

Imagen 2.17 Test 6

Se espera a que vuelva a lockear y se crea hilo_timeout para en caso de que no suceda luego del reset, la simulación termine.

```
--------- TEST 6: RESETS ASESINOS ---------
[6081185000 ns] Valid #1 | Generador: ff3b | Recibido: ffd3 | Checker: 0000 | MISMATCH | Matches totales: 0  |  Lock: 0
[6081215000 ns] Valid #2 | Generador: fe5b | Recibido: ff8b | Checker: ff8b | MATCH    | Matches totales: 1  |  Lock: 0
[6081235000 ns] Valid #3 | Generador: f91b | Recibido: ff3b | Checker: ff3b | MATCH    | Matches totales: 2  |  Lock: 0
[6081285000 ns] Valid #4 | Generador: 201b | Recibido: fe5b | Checker: fe5b | MATCH    | Matches totales: 3  |  Lock: 0
[6081305000 ns] Valid #5 | Generador: 806c | Recibido: fc9b | Checker: fc9b | MATCH    | Matches totales: 4  |  Lock: 0
[6081315000 ns] Valid #6 | Generador: 00f5 | Recibido: f91b | Checker: f91b | MATCH    | Matches totales: 5  |  Lock: 0
[6081315000 ns] Sistema lockeado. Operando con normalidad...
[6081325000 ns] LOCK: El estado de lock cambió a 1
[6081345000 ns] Valid #7 | Generador: 01ea | Recibido: f21b | Checker: f21b | MATCH    | Matches totales: 6  |  Lock: 1
[6081385000 ns] Valid #8 | Generador: 07a8 | Recibido: e41b | Checker: e41b | MATCH    | Matches totales: 7  |  Lock: 1
[6081398000 ns] ---> INYECTANDO RESET
Reset Checker.
[6081399000 ns] Deslockeado con éxito. El checker reaccionó al reset.
[6081405000 ns] UNLOCK: El estado de lock cambió a 0
[6081435000 ns] Valid #9 | Generador: 07a8 | Recibido: c81b | Checker: 0000 | MISMATCH | Matches totales: 7  |  Lock: 0
[6081455000 ns] Valid #10 | Generador: 1ea0 | Recibido: 901b | Checker: 0000 | MISMATCH | Matches totales: 7  |  Lock: 0
[6081515000 ns] Valid #11 | Generador: a8c3 | Recibido: 201b | Checker: 0000 | MISMATCH | Matches totales: 7  |  Lock: 0
[6081525000 ns] Reset liberado. Esperando que el sistema se recupere por sí solo...
[6081545000 ns] Valid #12 | Generador: a356 | Recibido: 4036 | Checker: 0000 | MISMATCH | Matches totales: 7  |  Lock: 0
[6081555000 ns] Valid #13 | Generador: a356 | Recibido: 806c | Checker: 806c | MATCH    | Matches totales: 8  |  Lock: 0
[6081575000 ns] Valid #14 | Generador: a356 | Recibido: 00f5 | Checker: 00f5 | MATCH    | Matches totales: 9  |  Lock: 0
[6081625000 ns] Valid #15 | Generador: 8d02 | Recibido: 01ea | Checker: 01ea | MATCH    | Matches totales: 10  |  Lock: 0
[6081655000 ns] Valid #16 | Generador: 68a4 | Recibido: 03d4 | Checker: 03d4 | MATCH    | Matches totales: 11  |  Lock: 0
[6081665000 ns] Valid #17 | Generador: d148 | Recibido: 07a8 | Checker: 07a8 | MATCH    | Matches totales: 12  |  Lock: 0
[6081665000 ns] EXITO: El sistema sobrevivió al reset y volvió a lockearse.
[6081675000 ns] LOCK: El estado de lock cambió a 1
[6081705000 ns] Valid #18 | Generador: 1571 | Recibido: 0f50 | Checker: 0f50 | MATCH    | Matches totales: 13  |  Lock: 1
[6081715000 ns] Valid #19 | Generador: 1571 | Recibido: 1ea0 | Checker: 1ea0 | MATCH    | Matches totales: 14  |  Lock: 1

=====================================================
 EL TEST 6 (RESETS RANDOMS) PASO EXITOSAMENTE.
=====================================================
```

Imagen 2.18 Salida por consola Test 6

Se ven 4 Mismatch ya que los primeros 3 ocurren mientras está persionado el reset, luego al soltarlo viene un Mismatch por el ciclo donde todavía no calculo el siguiente valor, y luego comienzan los matches hasta lockearse.

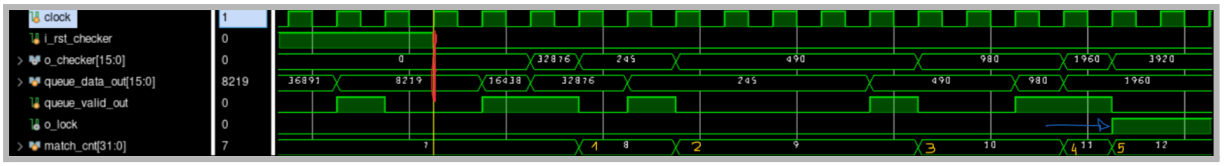


Imagen 2.19 Waveform Test 6

En esta imagen podemos ver la señal de reset cuando se suelta (es activo por alto). En ese momento el valor del o_checker es 0, luego del primer valid recibido se toma el valor 32876 como entrada (lo cual sigue reflejando un mismatch) y luego para el siguiente valid, ya el valor predicho es el mismo que el entrante por lo que da match 5 veces consecutivas y el valor de o_lock se pone en 1 lo que significa que se lockeo el sistema.

Test 7 Cambio Dinámico de Semilla

Verifica que el checker pueda re lockearse si el los datos generados por el generador cambian abruptamente a una secuencia distinta por un cambio de semilla y reset.

Una vez lockeado el sistema, al generador se le realiza un soft_reset cambiando la semilla, lo que cambia repentinamente la secuencia, el checker debe ser capaz de deslockearse y volverse a lockear como corresponde.

```
[24749694000 ns] ---> ACCION: Cambio de semilla
Puerto de Seed seteado a: b213
Soft Reset.
[24749725000 ns] Valid #15 | Generador: b213 | Recibido: 01ea | Checker: 01ea | MATCH    | Matches totales: 14  |  Lock: 1
[24749835000 ns] Valid #16 | Generador: b213 | Recibido: 03d4 | Checker: 03d4 | MATCH    | Matches totales: 15  |  Lock: 1
[24749885000 ns] Valid #17 | Generador: b213 | Recibido: 07a8 | Checker: 07a8 | MATCH    | Matches totales: 16  |  Lock: 1
[24749915000 ns] Valid #18 | Generador: b213 | Recibido: 0f50 | Checker: 0f50 | MATCH    | Matches totales: 17  |  Lock: 1
[24749945000 ns] Valid #19 | Generador: b213 | Recibido: 1ea0 | Checker: 1ea0 | MATCH    | Matches totales: 18  |  Lock: 1
[24749985000 ns] Valid #20 | Generador: b213 | Recibido: 3d40 | Checker: 3d40 | MATCH    | Matches totales: 19  |  Lock: 1
[24749995000 ns] Valid #21 | Generador: b213 | Recibido: 7a80 | Checker: 7a80 | MATCH    | Matches totales: 20  |  Lock: 1
[24750035000 ns] Valid #22 | Generador: b213 | Recibido: f500 | Checker: f500 | MATCH    | Matches totales: 21  |  Lock: 1
[24750065000 ns] Valid #23 | Generador: b213 | Recibido: ea2d | Checker: ea2d | MATCH    | Matches totales: 22  |  Lock: 1
[24750085000 ns] Valid #24 | Generador: b213 | Recibido: d477 | Checker: d477 | MATCH    | Matches totales: 23  |  Lock: 1
[24750095000 ns] Valid #25 | Generador: b213 | Recibido: a8c3 | Checker: a8c3 | MATCH    | Matches totales: 24  |  Lock: 1
[24750185000 ns] Valid #26 | Generador: b213 | Recibido: 51ab | Checker: 51ab | MATCH    | Matches totales: 25  |  Lock: 1
[24750235000 ns] Valid #27 | Generador: b213 | Recibido: a356 | Checker: a356 | MATCH    | Matches totales: 26  |  Lock: 1
[24750305000 ns] Valid #28 | Generador: b213 | Recibido: 4681 | Checker: 4681 | MATCH    | Matches totales: 27  |  Lock: 1
[24750425000 ns] Valid #29 | Generador: b213 | Recibido: 8d02 | Checker: 8d02 | MATCH    | Matches totales: 28  |  Lock: 1
[24771835000 ns] Valid #30 | Generador: 9001 | Recibido: 640b | Checker: 1a29 | MISMATCH | Matches totales: 28  |  Lock: 1
[24771865000 ns] Valid #31 | Generador: 80bc | Recibido: c816 | Checker: 3452 | MISMATCH | Matches totales: 28  |  Lock: 1
[24771875000 ns] Valid #32 | Generador: 80bc | Recibido: 9001 | Checker: 68a4 | MISMATCH | Matches totales: 28  |  Lock: 1
[24771885000 ns] UNLOCK: El estado de lock cambió a 0
[24771895000 ns] Valid #33 | Generador: 80bc | Recibido: 202f | Checker: d148 | MISMATCH | Matches totales: 28  |  Lock: 0
[24771925000 ns] Valid #34 | Generador: 80bc | Recibido: 405e | Checker: 405e | MATCH    | Matches totales: 29  |  Lock: 0
[24771955000 ns] Valid #35 | Generador: 80bc | Recibido: 80bc | Checker: 80bc | MATCH    | Matches totales: 30  |  Lock: 0
[24771985000 ns] Valid #36 | Generador: 02aa | Recibido: 0155 | Checker: 0155 | MATCH    | Matches totales: 31  |  Lock: 0
[24772005000 ns] Valid #37 | Generador: 02aa | Recibido: 02aa | Checker: 02aa | MATCH    | Matches totales: 32  |  Lock: 0
[24772115000 ns] Valid #38 | Generador: 552d | Recibido: 0554 | Checker: 0554 | MATCH    | Matches totales: 33  |  Lock: 0

[24772115000 ns] ---> EXITO: El sistema se volvio a lockear con el cambio de semilla.
[24772125000 ns] Valid #39 | Generador: aa5a | Recibido: 0aa8 | Checker: 0aa8 | MATCH    | Matches totales: 34  |  Lock: 1
[24772125000 ns] LOCK: El estado de lock cambió a 1
[24772135000 ns] Valid #40 | Generador: 5499 | Recibido: 1550 | Checker: 1550 | MATCH    | Matches totales: 35  |  Lock: 1
[24772165000 ns] Valid #41 | Generador: 5499 | Recibido: 2aa0 | Checker: 2aa0 | MATCH    | Matches totales: 36  |  Lock: 1
[24772175000 ns] Valid #42 | Generador: 5499 | Recibido: 5540 | Checker: 5540 | MATCH    | Matches totales: 37  |  Lock: 1
[24772185000 ns] Valid #43 | Generador: a932 | Recibido: aa80 | Checker: aa80 | MATCH    | Matches totales: 38  |  Lock: 1
[24772195000 ns] Valid #44 | Generador: 5249 | Recibido: 552d | Checker: 552d | MATCH    | Matches totales: 39  |  Lock: 1
[24772215000 ns] Valid #45 | Generador: a492 | Recibido: aa5a | Checker: aa5a | MATCH    | Matches totales: 40  |  Lock: 1
[24772255000 ns] Valid #46 | Generador: 9212 | Recibido: 5499 | Checker: 5499 | MATCH    | Matches totales: 41  |  Lock: 1
[24772275000 ns] Valid #47 | Generador: 2409 | Recibido: a932 | Checker: a932 | MATCH    | Matches totales: 42  |  Lock: 1
[24772295000 ns] Valid #48 | Generador: 2409 | Recibido: 5249 | Checker: 5249 | MATCH    | Matches totales: 43  |  Lock: 1
[24772305000 ns] Valid #49 | Generador: 2409 | Recibido: a492 | Checker: a492 | MATCH    | Matches totales: 44  |  Lock: 1

============================================================
 EL TEST 7 (CAMBIO DE SEMILLA) PASÓ EXITOSAMENTE.
============================================================
```

Imagen 2.20 Salida por consola Test 7

Se puede ver como el sistema estaba lockeado, luego del cambio de semilla sigue matcheando ya que todavia hay datos en la cola hasta que la nueva secuencia llega a la cola y por lo tanto al checker. En este caso al ser una secuencia completamente distinta (en lugar de una rafaga de datos corruptos como en anteriores tests), el checker pierde su estado de lock, luego una vez deslockeado, se recompone con la nueva secuencia, funcionando correctamente.

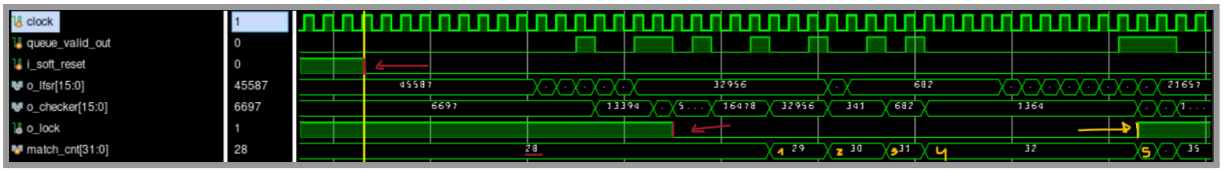


Imagen 2.21 Waveform Test 7

En las señales se ve como al caer el soft reset, el valor de match count se mantiene igual por lo que cae o_lock, y como luego de 5 matches consecutivos vuelve o_lock a valer 1 lo que indica que se lockeo correctamente.

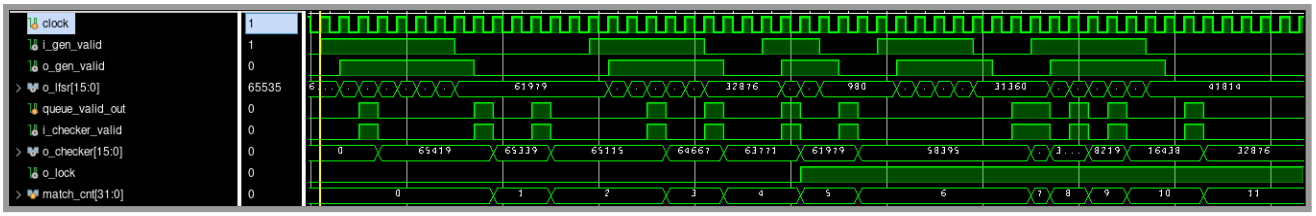


Imagen 2.21 Waveform general

En esta imagen podemos ver el comportamiento completo de las señales de interés.

- Vemos en primer lugar el clock habilitado.

- Vemos como tenemos a la entrada del generador una señal de valid, y como a la salida del generador tenemos el dato. La señal de valid que acompaña temporalmente a dicho dato, notar que esta un ciclo atrasada respecto a la señal valid de entrada. Ya que al generador le toma un ciclo de reloj calcular el dato siguiente de la secuencia.

- Se puede notar como la entrada de valid del checker *i_checker_valid* es la misma que que *queue_valid_out* ya que la salida de la cola es la entrada del checker como se mencionó anteriormente.

- Notamos como luego de un ciclo de que le llegue al checker la señal de valid, tenemos el nuevo valor generado por el checker ya que al checker también le toma un ciclo de clock calcular el dato esperado.

- Por último vemos como luego de 5 matches consecutivos se cambia a estado de LOCK ya que podemos ver *o_lock* en 1 lógico.

Implementación y Síntesis

Finalmente se adjuntan imágenes de la implementación del diseño y gráficas y tabla de recursos utilizados.

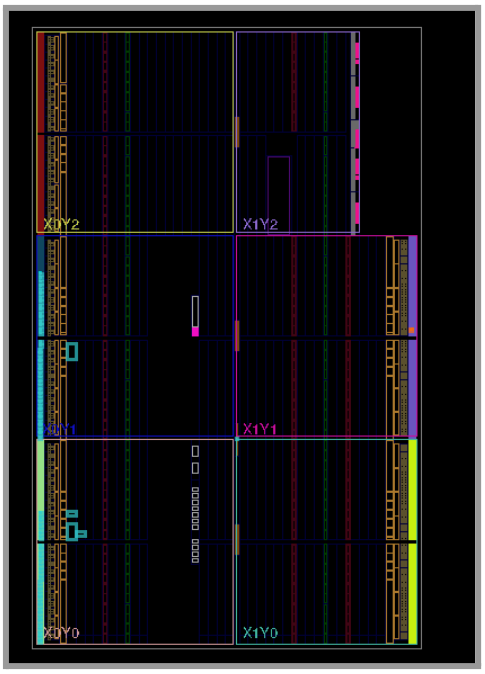
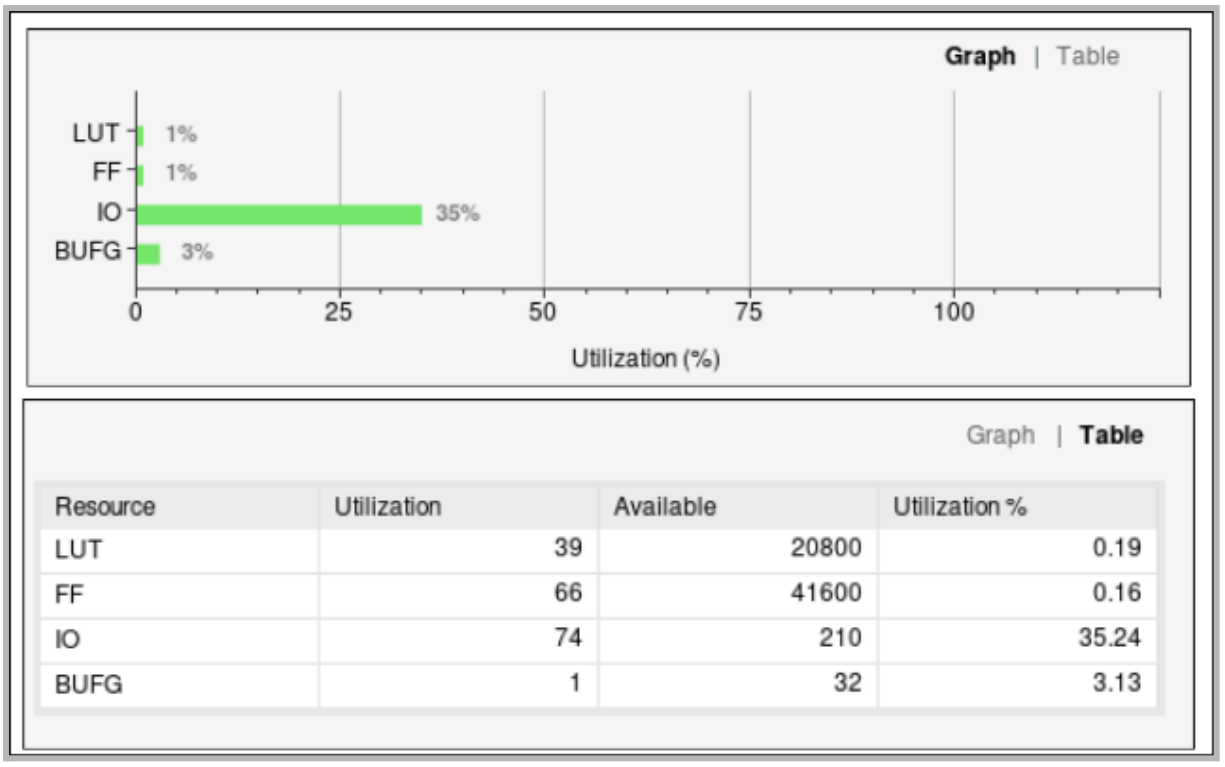


Imagen 2.21 Síntesis



Resource	Utilization	Available	Utilization %
LUT	39	20800	0.19
FF	66	41600	0.16
IO	74	210	35.24
BUFG	1	32	3.13

Imagen 2.22 Resumen utilización de recursos

Observaciones y aspectos a mejorar

- La máquina de estados si bien funciona correctamente, lo más óptimo debería ser separar su funcionamiento en una parte secuencial y una parte combinacional. De manera que la parte combinacional se encargue de decidir el próximo estado y la parte secuencial de actualizar los registros correspondientes.

- La generación de errores en la cola *queue.sv* se obtiene realizando una operación XOR. Esto solo genera un flip en el bit menos significativo. Si bien esto funciona ya que el dato queda corrupto, se podría hacer más caótico (por lo tanto mas cercano a la realidad) generando un tipo de error random en el bit.

- El diseño de la cola con una probabilidad de pop del 30% (lo cual fue realizado para agregar retardo aleatorio), de nuevo, si bien funciona correctamente, puede dar lugar a un crecimiento desmedido del tamaño de la cola. Para eso podría verse otro diseño donde el retardo no se genere de esta manera con probabilidad de extracción de datos.

El código completo del TP2 puede encontrarse en el ANEXO

Conclusiones

A lo largo de los dos trabajos prácticos se logró adquirir y consolidar conocimientos fundamentales sobre el diseño digital utilizando Verilog y el flujo de desarrollo sobre FPGA. El primer trabajo permitió introducirse en el lenguaje y en los conceptos básicos de diseño mediante la implementación del sistema de shifteo de LEDs, mientras que el segundo permitió profundizar estos conocimientos mediante el desarrollo del generador de números pseudoaleatorios LFSR de Galois y el circuito checker para a la verificación de la secuencia generada.

Además de la implementación de los sistemas, ambos trabajos permitieron poner en práctica conceptos relacionados con la estructura de un diseño en Verilog, como la utilización de módulos, señales, bloques combinacionales y secuenciales, la organización jerárquica del sistema y módulo top como punto de integración de los distintos bloques. También se adquirió experiencia en el entorno Vivado y en las distintas etapas necesarias para llevar un diseño desde su descripción hasta su simulación y posterior implementación.

Una parte importante estuvo relacionada con el proceso de testing y verificación. El desarrollo de los testbenches permitió comprender el comportamiento temporal del circuito, la sincronización con el reloj, los retardos asociados a los ciclos y el correcto acompañamiento temporal entre las distintas señales. Las dificultades encontradas durante las pruebas, especialmente aquellas relacionadas con ciclos de reloj y desfasajes temporales, permitieron comprender de forma más concreta la naturaleza paralela y sincronizada de los sistemas digitales.

En conjunto, los dos trabajos permitieron pasar de una comprensión principalmente teórica de los circuitos digitales a una experiencia práctica de diseño, simulación, depuración y verificación.

ANEXO

Aquí se encuentran los repositorios con el código completo de cada uno de los trabajos realizados. Se encuentran divididos en distintas carpetas RTL y TESTS. Dentro de RTL se encuentra el diseño. Dentro de TESTS tenemos los archivos de cada uno de los tests y el tesbtench.

- **Repositorio Trabajo Práctico 1**
- **Repositorio Trabajo Práctico 2**